

AI 通识课程平台一致性评估

横纵分析法深度研究报告

作者: Codex

评估时间：2026-07-15 | 评估对象：当前本地工作区、Git 提交基线与生产服务器 | 方法：一致性原理审计 + 横纵分析法 | 评估性质：只读取证，不修改业务代码与服务器

目录

- 1. 一句话结论
- 2. 评估方法与评分口径
- 3. 纵向分析：项目如何走到今天
- 4. 横向分析：当前七个一致性切面
- 5. 三层对照：规范、实现与运行态
- 6. 交汇判断：真正的优势与主要断裂
- 7. 整改路线与验收门
- 8. 三种未来情景
- 9. 证据、限制与信息来源

一、一句话结论

这个项目已经是一个**功能闭环较完整、局部测试保护较强、当前线上运行健康，但跨资源生命周期、缓存语义、发布可复现性和服务器安全/灾备尚未统一**的系统。

如果只看页面能否使用、接口是否有回归保护、当前服务有没有报错，它的完成度相当高；如果要求任意资源都遵循同一种删除语义、任意环境都能从一个提交重建、任意进程数下限流和缓存都保持同样结果、任意线上故障后都能恢复数据，项目还没有到“系统规则高度一致”的阶段。

本次加权总分为 **64/100**。这个分数不是功能完成率，而是“相同概念在不同层、不同入口、不同时间点是否仍表达同一件事”的评分。

维度	权重	得分	加权结果	判断
领域模型与产品语言	15%	72	10.8	主干清楚，共享题库语义仍借用课程归属
架构分层与职责边界	15%	60	9.0	服务层已形成，但 150 个路由中有 33 个直接操作数据库
数据生命周期与事务	20%	58	11.6	课程软删较完整，其他核心资源仍混用物理删

维度	权重	得分	加权结果	判断
权限、安全与身份	15%	76	11.4	角色与教师归属较稳，生产传输和多进程安全不足
接口、前端与交互	10%	82	8.2	统一响应、中文文案和静态契约表现较好
测试、文档与可追溯性	10%	71	7.1	测试强，文档事实分散且存在互相冲突
部署、可复现性与灾备	15%	39	5.9	当前健康，但脏工作区、无备份、磁盘与 HTTPS 风险突出
总计	100%		64.0	工程可用度高于系统一致性成熟度

二、评估方法与评分口径

2.1 什么叫“一致”

一致性不等于所有文件长得一样，也不等于所有接口都返回同一段 JSON。对这个项目，更有用的判断标准有七条：

- 单一事实源**：课程归属、题目归属、登录状态、部署版本、配置值，各自只能有一个可被信任的源头。
- 同语义同规则**：叫作“删除”的操作，无论从教师端、管理员端还是回收站进入，都要有可解释且稳定的生命周期。
- 路径无关性**：同一个业务结果不应因为走了不同 API、不同角色入口、不同 worker 或不同缓存状态而改变。
- 层次闭合**：页面类型、API 契约、Schema、服务、ORM 和数据库结构应能互相验证，而不是依赖口头约定。
- 时间一致性**：并发请求、重启、缓存过期、软删恢复和数据迁移之后，系统仍要保持合法状态。
- 环境可复现**：本地工作区、Git 提交、服务器源码和已发布静态文件应能映射到同一个发布版本。
- 承诺与能力相符**：文档写下的稳定事实，必须能在代码或服务上找到证据；计划中的未来能力不能提前写成当前能力。

可以把它想成一个交换图：从“业务规范”走到“服务器结果”，无论经过前端还是直接调用 API，无论命中缓存还是查询数据库，最终都应该落在同一个合法状态。任何一条路径得出不同结果，那里就是一致性断点。

2.2 取证范围

本次没有修改业务代码或服务器。取证覆盖：

- 知识图谱 `graphify-out/graph.json` 的跨文件查询；
- `AGENTS.md`、项目地图、前后端 README、实施计划和修改记录；
- FastAPI 路由、服务、Schema、ORM、兼容初始化、缓存与 Redis 封装；
- Vue 路由、身份状态、Axios 统一层、API 模块和生产构建；
- 本地全量后端测试、前端 42 个静态契约、类型检查和 Vite 构建；
- 当前生产服务器的 Git 状态、systemd、Nginx、MySQL、Redis、磁盘、Swap、journal、上传目录、端口、健康检查、构建资产与备份状态。

这里刻意区分三份对象：

- **Git 基线**：`main` 当前提交 `6155d701...`，提交时间 2026-07-12。
- **本地工作区**：在同一提交之上存在大量未提交改动，覆盖软删读路径、共享题库、学习页、Nginx 与部署脚本。
- **生产服务器**：也在同一提交之上，但只有另一组未提交后端改动和独立发布的前端产物。

三者不是同一个版本。这是后文多个问题的共同根。

三、纵向分析：项目如何走到今天

3.1 从功能集合到课程聚合根

项目早期最重要的演进，是把课程确定为教师端业务的归属根。班级、资料、题目、作品、课时和进度逐步围绕课程组织，教师权限也从“角色是 teacher”收紧到“资源属于当前 teacher”。这一选择让后台管理结构变得清楚：教师端导航、课程筛选、班级选课和成绩统计都能沿课程向下寻找。

现在的稳定事实仍能看到这条主线：`Course -> Class -> StudentClassEnrollment` 负责学生课程访问，`Course -> Material` 负责资料，`Course -> Lesson` 与两类进度表负责学习过程，作业则通过 `Announcement -> AnnouncementClass` 选择班级，再由 `QuizAttempt.announcement_id` 固定答题事实。课程聚合根不是文档里的口号，它已经进入模型、权限查询和测试。

这也是项目最牢固的一致性来源。

3.2 教师数据隔离成为第二条主线

随着教师端页面增多，资源归属不再只靠前端隐藏按钮。服务层普遍加入 `created_by`、`teacher_id` 或教师课程过滤，学生访问课程又集中到 `student_can_access_course()`。近期修复还把未加入课程详情、自由练习越权提交、私有题答案提前下发等问题收紧到后端。

这条演进的价值在于：权限逐渐从“界面约束”变为“数据查询约束”。当前 `require_role()`、`require_roles()`、`get_current_user()` 和资源归属过滤形成了相对稳定的身份骨架。教师端危险操作有确认、失败有中文提示，接口拒绝不再只靠按钮是否显示。

不过，权限规则仍分散在路由、多个服务和若干专用 helper 中。只要新入口绕开既有服务，开发者仍需记得重新写一遍资源归属条件。它已经比早期安全很多，却还没有达到“默认安全、遗漏困难”的程度。

3.3 共享题库改变了“题目属于课程”的含义

公共课程题库从复制模型改成全站共享，是一次影响很深的领域调整。现在教师和管理员新增题目会记录 `created_by` 与贡献日志；教师编辑权看题目创建人；管理员可以从任意有效公共课程入口管理全站活跃题目；课程删除前还需要把题目转挂到其他课程，避免随课程消失。

问题不在共享题库这个方向，而在数据模型仍用 `Question.course_id` 承担物理挂载。对课程详情来说，题目数已经是“全站活跃题目数”；对管理入口来说，所选公共课程只是进入共享题库的门；对删除来说，`course_id` 又决定必须先把题目迁走。

同一个字段同时承担“保存位置”“历史来源”“管理入口上下文”三种含义。代码通过 `created_by`、`question_bank_root_course_id`、贡献日志和转挂逻辑补足语义，功能可以成立，但理解成本和边界条件持续增加。早期“题目直接挂课程”的好决定，在共享题库阶段开始产生包袱。

3.4 大文件预览推动文件链路成熟，也暴露运行约束

大文件预览试点带来了相当扎实的一轮工程化：`StoredFile`、资料专用文件接口、短时签名 URL、Nginx `X-Accel-Redirect`、Range 206、PDF worker MIME、公开与私有文件权限区分、前后端共用站内预览。这部分有真实文件验收，也有静态回归保护。

它让文件访问从“把 uploads 暴露出去”转向“后端先授权，Nginx 再传输”，方向正确。服务器当前 `/data/tongshi/uploads` 权限为 `750`，归属 `ubuntu:nginx`，Nginx 内部路径使用 `internal`，44 个文件共约 29MB，链路与设计基本一致。

但上传上限、前端超时和服务器能力没有一起收口。后端与 Nginx 允许 1GB，前端 Axios 默认超时只有 10 秒，实际 Nginx 读写超时为 120 秒，本地样例写 300 秒；服务器只有 2 核、3.4GB 内存和 3.3GB 可用磁盘。文件链路的“鉴权一致性”已经较好，“容量与超时一致性”仍未完成。

3.5 进度、软删除、通知与审计把系统推向平台化

7 月上旬加入课时进度、软删除、回收站、通知中心和审计日志，项目从课程功能集合走向平台基础设施。这里的设计 ambition 很高：进度使用原子 upsert，通知有偏好和过期时间，审计日志覆盖资源操作，七类核心实体增加 `deleted_at/deleted_by`。

难点也从这里开始。增加两个删除字段很容易，统一所有写入口、读入口、统计口径、缓存键、文件权限和恢复规则却需要跨越整个系统。当前课程正常删除已经调用 `soft_delete()`，并能级联班级、资料 and 作业；班级、资料、作业、作品和若干管理员公共资源仍直接 `db.delete()`。有些读路径已经过滤软删，有些公告列表、详情和未读计数仍没有过滤。

这不是一个孤立 bug，而是一次平台化改造尚未完成迁移的典型中间状态：新机制已经存在，旧入口还活着。

3.6 近期阶段修复提高了局部正确性，发布治理没有同步升级

最近几轮修复围绕学生权限、软删读路径、共享题库、公开学习和文件链路，测试数量随之增长。本次本地验证中，330 个后端用例在全量运行中通过，5 个因审计命令的临时目录父路径不存在而未完成 setup；改用系统临时目录后，这 5 个用例全部通过。另有 1 个真实 MySQL 条件用例因未配置测试库跳过。前端 42 个静态测试、`vue-tsc` 和生产构建全部通过。

这证明代码修改的局部闭环能力已经成熟。

发布治理却仍保留手工时代的痕迹：本地和服务器都在同一个旧提交之上直接积累未提交改动；服务器 `frontend/dist/index.html` 与 `/var/www/tongshi/index.html` 引用不同构建哈希；服务器仓库里的 Nginx 样例、当前本地样例和 `/etc/nginx/conf.d/tongshi.conf` 三者哈希均不同。项目越能快速修问题，发布源越容易分裂，这正是当前阶段最需要处理的矛盾。

四、横向分析：当前七个一致性切面

4.1 领域模型与产品语言：主干清楚，两个语义热点

课程、班级、学生选课、资料、课时、作业和答题事实的关系总体清楚。项目地图明确班级必须属于课程，学生权限通过活跃班级关联课程，答题统计以 `QuizAttempt` 为事实源。教师端中文命名也已经从“任务发布”“学生数据”等早期词汇收敛到“发布作业”“学生成绩”“作业完成”。

两个热点仍在制造歧义。

一个是“公告”模型承担作业。代码文件和实体叫 `Announcement`，页面叫发布作业，接口说明又使用“发布题目”“题目任务”。同一对象在代码、API 和界面有三套名称。它没有直接破坏功能，却让新开发者很难迅速判断公告、作业和任务是不是同一对象，也容易让删除与通知逻辑出现漏网入口。

另一个是共享题库仍借用课程外键。当前规则已经不是“一门课程拥有一组题”，而是“题目全站共享，课程为挂载与入口上下文，创建人决定编辑权”。继续让 `course_id` 看起来像传统所有权字段，会不断诱发错误判断。

结论：领域主干可维护，但应把“作业”和“共享题库”的真实语义正式写进模型边界，而不是只靠注释和修复记录维持。

4.2 架构分层：方向一致，事务边界分裂

项目规范要求 Routes -> Services -> Models。代码已经有较完整的 services 目录，核心权限、课程响应、题库、进度、文件和软删除都有服务承接。统一响应与统一业务异常也位于 core 层。

静态扫描得到另一个数字：150 个路由函数中，33 个仍直接调用 `db.query/add/delete/commit/flush/refresh/rollback`。集中区域包括管理员教师管理、公共课程阶段、课时、进度、展示内容、通知偏好和审核审批。

这里的影响不只是“风格不够整齐”。当部分服务自己提交、部分路由在服务返回后提交、部分批处理在路由里回滚时，事务边界无法从架构层一眼判断。未来要把审计日志、缓存失效、软删和业务写入放进同一个事务，就不得不逐入口核对。

结论：分层已经形成，尚未成为强约束。应先统一写事务边界，再逐步搬迁高风险路由；不需要为了纯度一次性重构所有读取接口。

4.3 数据生命周期：项目当前最大的业务一致性缺口

七类核心资源都具备软删除字段，回收站也能列表、恢复和彻底删除。课程正常删除已经转入软删并记录审计日志。这是新规则。

旧规则仍在以下路径存在：

- `class_service.delete_class()` 物理删除班级，并物理清理孤立作业及关联记录；
- `material_service.delete_material()` 直接删除资料；
- `announcement_service.delete_announcement()` 直接删除作业；
- `project_service.delete_project()` 直接删除作品、图片和点赞；
- 管理员公共课程、公共资料、共享题目删除仍包含物理删除；
- 教师删除与回收站 purge 对同一类实体可能产生不同生命周期。

读路径也没有完全同频。`task_service` 已对作业、班级和课程过滤软删，`announcement_service` 的列表、详情和未读统计却仍直接查询关联数据，没有统一过滤 `Announcement.deleted_at`、`Class.deleted_at` 与 `Course.deleted_at`。课程级联软删作业后，某些公告入口仍可能把它读出来。

恢复语义还有一个天然边界：课程软删级联只恢复“删除时间和删除人都与课程相同”的子资源，这个设计能避免误恢复此前独立删除的数据，是正确的。但普通班级/资料/作业删除没有统一走这套时间戳规则，回收站就无法成为全系统唯一删除事实源。

结论：软删除不是“缺几个过滤条件”，而是迁移未完成。阶段 D Task 1 仍应视为最高业务优先级。

4.4 权限与安全：应用权限较稳，运行边界偏弱

应用层的正面证据很多：

- 角色依赖集中在 `security.py`；
- 学生课程访问依赖活跃选课、活跃班级和活跃课程；

- 学生拉题不预发答案，提交范围与选课/公共共享题库对齐；
- 教师资源过滤普遍使用创建人或课程归属；
- 文件访问使用短时签名或后端授权，不把普通 JWT 放入 URL；
- 密码写路径、强制改密、审计日志已有较多回归测试。

运行边界存在三处明显落差。

生产只监听 80，没有 HTTPS。登录口令和 JWT 因而通过明文 HTTP 传输。后端进程监听 `0.0.0.0:8050`，Ubuntu UFW 处于 inactive；从外部访问 8050 当前会超时，说明云安全组很可能挡住了端口，但主机和进程自身没有形成第二道限制。systemd 完全可以把 Uvicorn 绑定到 `127.0.0.1`，与 Nginx 反代模型保持一致。

忘记密码错误次数仍保存在进程内字典。单 worker 下语义成立，一旦按 README 启动 4 worker，每个进程都有独立计数，限流会被分裂。Redis 服务虽在运行，这条安全状态并没有进入 Redis。

结论：权限逻辑适合当前单 worker、同机代理形态；HTTPS、环回绑定和跨 worker 限流完成前，不应把多 worker 写成已可用生产方案。

4.5 接口与前端：用户体验一致性是当前强项

152 条 FastAPI 路由没有发现相同 method/path 的重复注册。业务成功响应统一为 `code/data/message`，`BusinessException`、SQLAlchemy 异常和未捕获异常都被包装为同一业务形状；文件流和 Excel 下载作为明确例外返回原始响应。前端 Axios 拦截器统一解包 `data`，处理业务 401、HTTP 401、422 中文化和静默错误。

前端路由对游客、学生、教师、管理员有清楚的入口规则，过期 token 会清理，本地存储用户对象也做了结构校验。教师端固定导航、危险操作、中文空状态和文件预览都有静态契约保护。42 个静态测试、类型检查和生产构建通过，说明界面约定被持续维护。

契约层仍有一个长期风险：152 条路由没有任何显式 `response_model`。后端大量返回手工 dict，前端类型也手工维护。当前没有重复路径，不能等同于返回字段不会漂移。构建能证明 TypeScript 内部自治，无法证明 TypeScript 接口与线上 JSON 完全同构。

另一个运行差异是超时：Axios 全局 10 秒，普通接口合适，1GB 上传不合适；Nginx 实际 120 秒，本地样例 300 秒，后端上传又允许 1GB。上传进度条不能消除客户端 10 秒主动中止。

结论：前端表现层稳定；下一步应补契约生成/契约测试和按业务类型区分超时，而不是继续堆静态字符串断言。

4.6 缓存、数据库与并发：配置存在，语义没有闭合

服务器 Redis 正常响应，内存占用约 851KB，当前 DB size 为 0。代码中公共课程、课程详情和班级列表声明了缓存装饰器，但实现存在三组不相容：

- 默认键把函数的所有参数转成字符串，包含每次请求都不同的 SQLAlchemy Session 对象；

- 装饰器前缀中的 `{teacher_id}`、`{course_id}` 不会被格式化，而失效调用使用已经填值的 `classes:teacher:{真实ID}:*` ；
- 公共课程列表返回 ORM 对象、课程详情返回含 ORM 的 tuple，`json.dumps()` 无法序列化，异常后会静默降级查询数据库。

所以 Redis“服务可用”不等于缓存“工作正常”。当前更接近一个失败开放、基本不命中的缓存层。它暂时减少了脏缓存风险，却没有提供计划中的减压能力。仓库中还没有 `test_cache_behavior.py`，阶段 D 对缓存键、故障冷却和双进程限流的测试仍是计划。

数据库结构有强有弱。线上 MySQL 有 30 张表，当前 ORM 声明 29 张，所有 ORM 表和列都存在，额外的 `activity_events` 是历史表；列结构没有发现漂移。MySQL 开启 `binlog`，`innodb_flush_log_at_trx_commit=1`、`sync_binlog=1`，单机事务耐久参数较严格。

索引同步没有完成。线上缺少 22 个当前 ORM 已声明的索引，涉及七类软删字段、题干哈希、题目创建人、课程共享根、答题复合查询和通知分类/优先级/过期时间。`main.py` 启动时仍在锁外先 `create_all()`，再执行兼容逻辑；多 worker 同时启动时没有 MySQL 命名锁保护。

结论：当前单 worker、低数据量下数据库可用；启用 Redis 或多 worker 之前，应先完成阶段 D 的缓存与启动锁，而不是只改启动参数。

4.7 文档、部署与服务器：当前健康，重建能力不足

文档数量已经很大：23 份 plans、13 份 specs，`project-map.md` 366 行，后端修改记录 3672 行。详细记录保留了演进原因，这是优点。问题是“当前事实”散落在 AGENTS、项目地图、README、计划开头和长修改记录中。

直接冲突包括：

- `backend/README.md` 建议生产 `--workers 4`，单机加固计划和实际 `systemd` 明确保持单 worker；
- 后端修改记录声称存在 `REDIS_ENABLED`，当前配置类没有这个字段；
- `deploy/redeploy-server.ps1` 与部署 README 使用 `/var/www/tongshi`，本地 `deploy/nginx.conf` 却写 `/var/www/tongshi/frontend`；
- 本地 Nginx 样例使用 300 秒超时并关闭 request buffering，线上为 120 秒且缺少该项；
- 线上 `.mjs` 规则匹配全站，本地改动已收窄到 `/assets/`，但尚未部署。

生产服务器当前是“健康但脆弱”：

项目	当前实态	一致性判断
CPU / 内存	2 核 / 3.4GB，可用内存约 2.1GB	单 worker 合理
Swap	4GB，当前未使用， <code>swappiness=0</code>	有缓冲，但与计划值 10 不同

项目	当前实态	一致性判断
根盘	40GB 已用 92%，仅余 3.3GB	高风险，且继续恶化
journal	3.9GB，无限额配置	单项已大于根盘剩余空间
服务	Nginx、MySQL、Redis、后端均 active	当前运行健康
后端	自 7 月 14 日启动，0 次重启，无 warning 级日志	短期稳定
数据	MySQL 约 1.66MB，上传约 29MB / 44 文件	规模尚小，适合立即建立备份
备份	无 <code>/data/tongshi/backups</code> ，无 Tongshi timer	无可验证恢复点
网络	HTTP 80；后端监听全网卡 8050；外部 8050 当前超时	依赖云安全组，缺 HTTPS
发布	服务器仓库脏、静态发布物与仓库 dist 不同	无法从提交唯一重建
Nginx	实际配置、服务器仓库样例、本地样例三份不同	配置事实源分裂

服务器日志没有显示当前故障，静态页引用的 9 个资产也都存在。正因为现在还能平稳运行，治理动作应在事故前完成，而不是等磁盘或发布回滚失败之后再补。

五、三层对照：规范、实现与运行态

5.1 已经形成闭环的部分

稳定规则	规范层	实现层	运行/验证层	结论
统一业务响应	AGENTS 要求 <code>code/data/message</code>	<code>response.py</code> 与全局异常处理统一包装	前端拦截器统一解包，150 个路由未见普通 dict 例外	闭环较好
教师资源归属	教师只能访问自己的课程及下属资源	多数服务按 <code>created_by/teacher_id</code> 过滤	权限与资源范围测试通过	闭环较好

稳定规则	规范层	实现层	运行/验证层	结论
学生课程访问	必须通过活跃选课访问课程	<code>student_can_access_course()</code> 关 联班级与课程软删状态	未加入课程与越权练习回归测试通过	闭环较好
学生不预拿答案	拉题不下发答案，提交后返回判定	题目和作业路由按角色隐藏答案	练习流程静态与后端测试通过	闭环较好
文件私有访问	新文件走 <code>file_id</code> ，后端鉴权后传输	签名 URL、文件权限、X-Accel、Range	线上内部 alias 与目录权限正确，资产可用	闭环较好
前端中文与构建	文案中文、教师端稳定易扫	统一路由标题、组件和错误提示	42 个静态测试、类型检查、构建通过	闭环较好
ORM 列结构	模型与兼容层共同维护旧库	<code>create_all + schema_compat</code>	线上无缺表、无缺列	当前闭环

这些闭环解释了为什么系统现在可以稳定使用。它们不是表面完成度，而是规范、代码和测试三者已经对上。

5.2 只闭合了两层的部分

规则	已闭合	未闭合	后果
软删除	模型字段、回收站、课程删除、部分读过滤	班级/资料/作业/作品/公共资源写入口与公告读入口	同名“删除”产生不同生命周期

规则	已闭合	未闭合	后果
Redis	服务已安装、连接可用、代码有装饰器	键构造、序列化、失效、测试、限流共享	线上 Redis 空闲，扩 worker 后安全语义分裂
数据库兼容	表列对齐	22 个声明索引缺失，启动无全局锁	数据增长或多 worker 启动时风险放大
发布脚本	拉取、构建、rsync、重启、直接健康检查已有脚本	服务器脏工作区、静态产物无版本标记、Nginx/systemd 不完全归仓库	不能证明线上由哪个提交重建
运维计划	资源现状和任务写得很具体	journal、备份、磁盘 timer、Redis 依赖均未执行	计划不能提供运行保护
接口类型	前端内部类型检查、统一响应成立	152 条路由无 response model，前后端类型不自动对照	字段漂移只能靠人工和静态断言发现

5.3 三层都没有闭合的部分

发布版本标识是最明显的一项。当前没有一个 `/version` 或构建元数据能同时返回 Git SHA、构建时间和数据库兼容版本；前端静态文件也没有发布清单与后端版本绑定。本地、服务器源码和 `/var/www` 只能靠文件哈希人工比对。

可恢复性也是空白。MySQL binlog 能提高事务耐久性，但不是备份；上传文件和数据库都在同一块系统盘，服务器没有 Tongshi 备份目录、定时器和恢复演练。规范层有计划，实现层和运行层尚不存在。

端到端就绪检查尚未形成。`/health` 只返回固定版本与 ok，不检查 MySQL、Redis、上传目录可写、磁盘阈值或 Nginx 文件链；部署脚本访问后端直连地址，也没有验证 Nginx、前端资产和外部入口。当前健康依赖人工组合多条命令判断。

六、交汇判断：真正的优势与主要断裂

6.1 优势来自哪里

项目今天最大的优势不是技术栈，而是持续把发现的问题写成测试和阶段记录。学生权限、作业完成、文件预览、教师端中文文案、移动布局、共享题库等都经历了“问题—计划—实现—回归”的小闭环。前端 42 个静态契约和后端 300 多个用例，是这种工作方式积累出来的。

课程聚合根也是一个早期选择带来的长期收益。教师端可以沿课程划定数据边界，学生沿班级进入课程，资料和课时沿课程组织。只要新的功能尊重这条主线，它通常能较快融入现有结构。

统一业务响应和集中身份依赖则降低了跨端协作成本。前端可以把大多数错误处理放进一个拦截器，不必每个页面重新解释后端异常；后端也不混入多套错误包装。这一层已经有平台气质。

6.2 劣势的历史根源

当前主要劣势都能追溯到“快速功能迭代优先、系统治理后补”的路线。

软删除是在已经存在大量物理删除入口之后加入的，所以字段和回收站先到位，统一写路径最慢。共享题库是在题目已挂课程的模型上演进的，所以不得不增加根课程、创建人、贡献日志和转挂。部署脚本是在服务器已有手工工作区之后补的，所以它能自动执行步骤，却不能消除脚本接管前留下的脏状态。Redis 也是先有客户端和装饰器，再补正确性设计。

这些都不是错误选择。它们在早期帮助项目更快得到可用功能。代价是进入平台化阶段之后，必须安排一次“消除双重语义”的治理，而不能继续只修页面可见问题。

6.3 当前最危险的不是某个 bug，而是组合风险

单独看，根盘 92% 还没满；journal 3.9GB 还能写；服务器没有备份但数据量很小；Git 工作区虽脏，服务也能运行；HTTP 没有 TLS，但云安全组挡住了 8050。

把它们串起来，风险形态就变了：一次临时热修需要重新构建，构建占用剩余磁盘；发布物无法映射到干净提交，失败后不容易恢复上一版；日志或 Docker 继续增长可能把根盘推满；即使服务恢复，数据库和上传文件也没有可验证备份。这里没有一个单点必然导致事故，但多个薄弱环节会在同一次变更中互相放大。

我的判断是，项目已经过了“多做几个功能就更完整”的阶段。接下来最值钱的工作，是让每个关键状态只存在一份、每次发布都能重建、每次删除都能解释、每次故障都有恢复点。

七、整改路线与验收门

7.1 P0：先保护生产事实，建议 48 小时内完成

P0-1 建立数据库与上传文件备份，并做一次恢复验证

执行顺序应是备份、校验、恢复演练、再进行任何清理或部署。至少包含 MySQL 全量备份、`/data/tongshi/uploads` 归档、校验和、保留策略和失败告警。MySQL binlog 可以作为增量恢复辅助，不能替代全量备份。

验收门：能在隔离目录或测试库恢复，恢复后的表数量、关键记录计数、上传文件数量和抽样文件哈希一致；定时器失败时有非零退出码和可见日志。

P0-2 控制根盘与 journal

落实已有单机加固计划中的 journal 限额、磁盘阈值检查和安全清理。Docker 当前约 3.6GB，其中约 954MB 可回收；是否清理由运维逐项确认。不能在无备份的情况下删除未知目录或历史项目。

验收门：根盘回到安全水位，建议低于 80%；journal 有明确上限；磁盘检查 timer 生效；低于预留空间时部署脚本拒绝执行。

P0-3 固定网络边界

为用户入口配置 HTTPS；Uvicorn 改绑 `127.0.0.1:8050`；保留云安全组对 8050、3306、6379 的外部阻断，并把主机侧限制作为第二道边界。对登录、文件签名和普通 API 做 HTTP 到 HTTPS 跳转验证。

验收门：80 仅跳转，443 提供有效证书；公网无法直连 8050/3306/6379；Nginx 仍能代理 API 与 Range 文件流。

P0-4 选定唯一发布事实源

暂停继续在服务器工作区手工改代码。先把服务器四个已修改后端文件、本地当前改动和已发布前端做三方对照，确认哪些改动应进入 Git；服务器环境文件保留在版本外，其余源码必须回到可追溯提交。每次构建写入 Git SHA 与构建时间，发布脚本在脏工作区时默认拒绝部署。

验收门：服务器源码工作区除受控环境文件外干净；`/var/www` 与一次脚本构建产物哈希一致；应用版本接口能返回发布 SHA；能用同一提交在空目录重建。

7.2 P1：统一系统语义，建议 1—2 个迭代完成

P1-1 完成软删除统一入口

按阶段 D Task 1 把班级、资料、作业、作品和需要进入回收站的公共资源迁入统一软删服务。明确哪些关联记录随主资源软删、哪些事实记录保留、哪些只能 purge 时物理清理。公告列表、详情、未读数、教师列表和统计必须共用活跃过滤规则。

验收门：七类资源的普通删除都能在回收站看到；删除后所有正常读取、文件访问和统计不可见；恢复后只恢复同一批次级联项；purge 才发生物理删除；每条路径写审计日志。

P1-2 重做或暂时撤下缓存层

缓存键必须只由稳定业务参数组成，不包含 Session；键前缀、构造和失效使用同一函数；缓存值先格式化为 JSON 数据结构；通配失效用 `SCAN`，不使用阻塞式 `KEYS`；加入 Redis 故障冷却和命名空间。若短期不打算完成，移除无效装饰器比保留“看起来有缓存”更诚实。

忘记密码限流应使用 Redis 原子计数，单 worker 内存实现只作为明确降级。

验收门：连续请求出现可观测命中；新增/更新/删除后旧键消失；两个进程共享限流；Redis 关闭时业务可用且日志不过载；测试不接触生产 Redis DB。

P1-3 完成生产索引与启动锁

把 `create_all()` 和兼容升级放进同一个 MySQL 命名锁，补齐线上缺少的 22 个声明索引。升级前备份，在测试库验证，再以单实例执行升级。不要直接用多 worker 让多个进程竞跑 DDL。

验收门： ORM 声明索引与生产 inspector 对齐；两个并发初始化只有一个执行兼容升级；锁异常时启动失败而不是带半升级结构运行；MySQL 专用集成测试不再跳过。

P1-4 收敛写事务与高风险路由

先处理 33 个直接数据库操作路由中的写接口，把“业务写入 + 审计 + 缓存失效 + commit/rollback”放进服务层。读取型导出或简单查询可以后置，不追求一次性重构。

验收门： 高风险写路由不直接 commit；同一业务操作只有一个事务拥有者；异常路径能证明审计与业务数据不会一半成功。

P1-5 建立前后端契约检查

为核心分页、详情、登录、文件签名和作业接口增加显式响应 Schema，逐步从 OpenAPI 生成或校验 TypeScript 类型。文件流、Excel 和 keepalive 请求保留明确例外。

验收门： 核心接口字段删改会在 CI 失败；分页形状只有一种；前端无需用 `any` 掩盖契约差异。

7.3 P2：降低长期认知成本

P2-1 给共享题库独立领域身份

可以保留现有表，也可以后续引入 QuestionBank/Scope，但要让模型清楚表达：课程是题目挂载上下文，创建人是编辑归属，共享范围是全站。避免继续把课程外键当作所有权。

P2-2 统一“作业”语言

产品文案已经统一为作业，代码可以分阶段将公告服务中的业务术语改成 assignment/task，或至少建立正式术语表，说明 `Announcement(type=quiz)` 就是当前作业实体。不要在新接口继续增加第四套叫法。

P2-3 重构文档信息架构

建议只保留四类长期文档：当前架构事实、领域决策记录、运维手册、时间序列变更日志。计划文件保留历史，但计划开头不能再充当当前事实源。 `project-map.md` 不应继续无限追加修改记录；3672 行后端记录也应按年份或主题建立索引。

P2-4 增加可观测就绪检查

保留轻量 `/health` 作为存活检查，新增受控 `/ready` 检查数据库、Redis降级状态、上传目录、磁盘阈值和 Schema 版本。外部发布验收还应检查首页、一个 API、一个受控文件 Range 请求和构建版本。

八、三种未来情景

8.1 最可能情景：低并发下继续稳定，维护成本缓慢上升

如果维持单 worker、当前数据量和人工发布方式，平台大概率还能稳定运行。测试会继续挡住许多页面和业务回归，服务器短期也没有资源瓶颈。每次新功能仍可能按现有方式顺利上线。

代价是每次改动都要同时记住软删新旧入口、共享题库转挂、三份 Nginx 配置和本地/服务器差异。维护成本不会突然爆炸，而是以“每次上线多核对一点”的方式累积。团队越依赖少数熟悉历史的人，单一事实源越难建立。

8.2 最危险情景：磁盘压力与不可复现发布叠加

根盘继续被 journal、构建产物或工具目录占用，一次发布在构建或 rsync 阶段失败。因为服务器源码和静态发布物都不对应一个干净提交，回退不能只切 Git SHA；因为没有数据库和上传备份，清理磁盘或重装服务也缺少安全边界。HTTP 与主机边界问题还可能让故障处理期间暴露更多攻击面。

这个情景的危险来自组合，而不是流量。它完全可能发生在用户不多的时候。

8.3 最乐观情景：用一次治理换来可扩展基础

先做备份、磁盘和发布事实源，再完成软删、缓存、索引和启动锁。到那时，多 worker 不再只是改一个参数：限流共享、Schema 初始化、连接池和缓存键都有验证；发布可由提交重建；线上任何资源删除都能解释和恢复。

项目会从“功能很多、修得很快”跨到“规则少而稳定”。这时再做多人并发、2G2 核调优或对象存储，新增复杂度会落在清楚的边界上，而不是继续叠在历史特例上。

九、证据、限制与信息来源

9.1 本次验证结果

- 后端全量运行：330 passed、1 skipped、5 setup errors；5 个 setup error 均由审计命令指定的临时目录父路径不存在引起。
- 对上述 5 个用例使用系统临时目录复验：5 passed。

- 跳过项：未配置专用 `TEST_MYSQL_URL` 的真实 MySQL 条件测试；不能据此宣称 MySQL 并发升级已验证。
- 前端静态测试：42/42 通过。
- 前端 `vue-tsc --build`：通过。
- 前端 Vite 生产构建：通过；保留现有大 chunk 提示，最大业务/供应商块包括 WangEditor、课程详情和 Element Plus。
- 服务器后端：active，当前进程 0 次重启，自 2026-07-14 17:13 启动后未检出 warning 级日志。
- 服务器 Nginx：最近 error log 无记录；当前首页引用的 9 个构建资产均存在。
- 外部探测：80 端口可访问；8050 外部连接超时。该结果只能证明当前网络路径受阻，云安全组具体规则未通过云控制台核验。

9.2 评估限制

- 没有使用生产账号登录教师、学生和管理员页面，也没有重置任何生产密码，因此未对线上三角色做写操作浏览器验收。
- 没有执行生产压测、1GB 上传或同步预览压力测试，避免占用仅余 3.3GB 的系统盘。
- 没有读取或输出任何密码、密钥和 token；环境文件只核对键名与非敏感配置形态。
- 没有修改 Nginx、systemd、数据库、Redis、备份、文件权限或服务状态。
- 没有在生产数据库执行迁移或恢复演练。
- 知识图谱对“计划文档”的连接强于领域代码，且本地 skill 版本 0.8.30、已安装包 0.8.33；图谱用于导航，不作为单独结论来源。

9.3 主要仓库证据

- `AGENTS.md`：项目规范、当前优先级与服务器影响要求。
- `docs/superpowers/project-map.md`：当前架构、核心业务关系和长期约定。
- `docs/superpowers/frontend-guidelines.md`：教师端命名、交互与验收底线。
- `docs/superpowers/plans/2026-07-10-remediation-phase-d-soft-delete-cache-db.md`：软删、缓存、索引与启动锁待办。
- `docs/superpowers/plans/2026-07-14-single-server-ops-hardening.md`：服务器资源、备份和运维计划。
- `backend/main.py`：启动建表、兼容初始化、全局异常和健康检查。
- `backend/app/core/response.py`：统一响应结构。
- `backend/app/core/cache.py` 与 `backend/app/db/redis_client.py`：缓存键、序列化、失效和 Redis 连接。
- `backend/app/services/soft_delete_service.py`：软删、恢复、purge 和课程级联。

- `backend/app/services/question_service.py` 、 `class_service.py` 、 `material_service.py` 、 `announcement_service.py` 、 `project_service.py` : 各资源真实删除与读取路径。
- `backend/app/services/access_control_service.py` : 学生课程访问边界。
- `backend/app/models/entities.py` 与 `backend/app/db/schema_compat.py` : ORM 与兼容初始化。
- `frontend/src/api/http.ts` : 统一请求、错误处理和 10 秒默认超时。
- `frontend/src/router/index.ts` 与 `frontend/src/stores/auth.ts` : 角色路由、token 过期和登录状态。
- `deploy/nginx.conf` 、 `deploy/redeploy-server.ps1` 、 `deploy/README.md` : 仓库部署约定。

9.4 服务器只读取证

访问时间为 2026-07-15, 使用已有 SSH 配置以 `ubuntu` 身份执行只读命令。证据来自:

- `systemctl show/cat` 、 `journalctl` 、 `ss` 、 `df` 、 `free` 、 `swapon` 、 `sysctl` ;
- `nginx -T` 与 `/etc/nginx/conf.d/tongshi.conf` ;
- `/home/ubuntu/tongshi_all_two` 的 Git 提交、状态和文件哈希;
- `/var/www/tongshi` 构建文件时间、哈希与资产存在性;
- SQLAlchemy Inspector 对生产 MySQL 表、列和索引的只读比较;
- MySQL 版本、耐久参数和库体积;
- Redis ping、内存和 DB size;
- `/data/tongshi/uploads` 的目录权限、文件数量和容量;
- Tongshi timer、备份目录与部署环境检查脚本。

9.5 方法论说明

本报告把横纵分析法适配为内部系统审计: 纵轴追踪项目从课程聚合、权限收紧、共享题库、文件链到平台基础设施的演进; 横轴在当前时点比较领域、架构、数据、权限、接口、文档和服务七个切面。两条轴交汇后, 判断哪些今天的优势来自历史积累, 哪些今天的包袱来自尚未完成的迁移。

9.6 服务器部署影响

本次只新增评估报告, 并执行本地测试、构建和服务器只读检查。**不需要服务器拉代码、重启服务、重新构建前端、执行数据库迁移、修改环境变量或调整 Nginx。** 报告中的 P0/P1/P2 均为后续建议, 执行前应分别立项、备份并按项目规范编写实施计划。